

Hashing Pipeline Explanation

W S M Fernando

SC/2020/11691

Contents

1. def get_significant_digits(val, n=5):	5
2. def safe_float_from_bytes(b: bytes, precision: int = 12) -> float:	6
Abnormal cases.....	7
Fixes:.....	8
Turning bytes into a float safely	8
Denormal protection	9
Avoid Zero mantissa	9
Custom value-range clamping	10
Small magnitude handling	10
Large magnitude handling	10
Formal Mathematical Proof of Non-Invertibility.....	11
Domain – Range cardinality mismatch.....	11
Information Erasure (Significand / Exponent Repair)	13
Why this does <i>not</i> impact final output security.....	15
3. def safe_float_from_bytes(b: bytes, precision: int = 12) -> float:	16
1. Every <i>float</i> = <i>Sign</i> × <i>Mantissa</i> × <i>2Exponent</i>	17
2. What you do when you strip the exponent.....	18
3. Why this drastically changes the number.....	19
4. The mantissa is the precision fingerprint of the number	19
5. Why the value becomes small and predictable	20
6. Why this is useful in your hashing method.....	20
Generate HMACs for min and max	21

Extract floats	21
Ensure separation	22
Round and return.....	22
4. normalize_multi_dim_x_values_to_custom_range	22
Loop over each vector.....	23
Normalization options.....	24
Linear.....	24
Log.....	24
Clipped.....	24
Tanh	25
Map to $[x_{min}, x_{max}]$	25
5. generate_user_param_range.....	25
6. normalize_param.....	26
Linear.....	27
Log.....	27
Clipped.....	28
Tanh	28
Security Notes.....	28
7. apply_plot_function	29
What the function does (high level).....	29
Mathematical representations	30
Formal tool: level sets and dimensions	30
plot_type = 0	31
plot_type = 1	32
plot_type = 2	33
plot_type = 3	34
plot_type = 4	35

plot_type = 5	35
General mathematical proofs (statements can used in the paper)	36
Why multi-dimensional unpredictable functions are better than single-dimensional $f(\mathbf{x})$	36
Single-dim $f(\mathbf{x})$ vulnerabilities	36
Why multi-dimensional approach is safer	37
Practical, provable security claims (to include in the paper)	37
8. def map_char_to_function_with_x	38
High-level Purpose.....	39
Generate plot type using HMAC	39
Generate raw p1 and p2 values	40
Normalize p1 and p2 into user-specific ranges	40
Select dimension count n (1 to max_dims)	42
Generate $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$ via independent HMACs.....	42
Final Output	43
9. def entropy	43
Count frequency of each byte	43
Total number of bytes	44
Compute Shannon entropy.....	44
10. def iterative_plot_transform.....	45
High-Level Purpose of This Function.....	47
Initial Setup.....	47
Loop over iterations	47
Prepare next iteration	47
Generate raw x-values for each character	48
Normalize raw x-values into user-specific range.....	49
Normalize AGAIN into safe math range	49
Apply the plot functions (core transformation).....	49

Global Avalanche Mixing.....	50
Convert accumulator → float values	53
Pack all floats into binary	55
Final Output (After preparation for the next iteration)	55
11. def compare_methods	56
High-Level Purpose	58
Salt generation	59
SHA256 Block.....	60
1. Start memory tracking:.....	60
2. Measure execution time:	60
3. Measure memory peak	60
4. Entropy.....	60
5. Length	60
6. Verification	60
bcrypt Block.....	61
How and Why?	61
Where is the salt in a bcrypt hash?.....	62
Why bcrypt does this automatically.....	62
scrypt Block	62
Argon2 Block.....	63
1. What Argon2 Is.....	64
2. What the Salt Does in Argon2	64
3. Internal Steps Argon2 Executes (Technical).....	66
4. What our Argon2 Hash String Contains	67
5. How Verification Works	67
6. Why Argon2 is perfect as the Final Layer of this Pipeline	68

1. def get_significant_digits(val, n=5):

```
def get_significant_digits(val, n=5):
    # Return the first n significant digits of a number as a list of integers.
    if val == 0:
        return [0]
    exponent = math.floor(math.log10(abs(val)))
    scaled = abs(val) / (10 ** exponent) # scale to [1,10)
    digits = []
    for _ in range(n):
        digit = int(scaled)
        digits.append(digit)
        scaled = (scaled - digit) * 10
    return digits
```

Purpose: Extract the first n meaningful digits from val, ignoring scientific notation, decimal points, or sign.

if val == 0:

return [0]

- Special-case: has no 0 definable significant digits via log10.
- Returns [0] so next functions don't fail due to empty list.

*scaled = abs(val) / (10 ** exponent) # scale to [1,10)*

- Normalize the number to the range [1,10)
- Example: 1234 → 1.234, 0.0045 → 4.5

digits = []

for _ in range(n):

digit = int(scaled)

digits.append(digit)

*scaled = (scaled - digit) * 10*

return digits

- Initialize list to hold extracted digits.
- Repeatedly:
 - Take integer part → first digit
 - Append digit to result list
 - Remove digit and shift left to next digit place

Example: 1234

Step	scaled	digit	new scaled
1	1.234	1	2.34
2	2.34	2	3.4
3	...	...	...

Returns list of integers, e.g. 1234 → [1, 2, 3, 4, ...]

2. def safe_float_from_bytes(b: bytes, precision: int = 12) -> float:

```
def safe_float_from_bytes(b: bytes, precision: int = 12) -> float:
    if len(b) < 4:
        # pad if necessary
        b = b.ljust(4, b'\x00')
    # Convert bytes to uint32 bits
    bits = int.from_bytes(b, byteorder='big')
    sign = (bits >> 31) & 0x1
    exponent = (bits >> 23) & 0xFF
    mantissa = bits & 0x7FFFFFFF
    # Clamp exponent to avoid denormals, zeros, Inf, NaN
    orig_exponent = exponent
    if exponent == 0 or exponent == 255:
        exponent = 127 # bias for exponent 0 → safe normal number
    # Clamp mantissa to avoid exact zero
    orig_mantissa = mantissa
    if mantissa == 0:
        mantissa = 1
    safe_bits = (sign << 31) | (exponent << 23) | mantissa
    safe_bytes = safe_bits.to_bytes(4, byteorder='big')
    val = struct.unpack('>f', safe_bytes)[0]

    # clamp
    min_val = 1e-12
    max_val = 1e12
```

```

abs_val = abs(val)

if abs_val < min_val:
    # Small value: clamp using max significant digit * min_val
    digits = get_significant_digits(val, n=5)
    max_digit = max(digits) if digits else 1
    val = math.copysign(max_digit * min_val, val)

elif abs_val > max_val:
    # Large value: clamp using max significant digit * max_val
    digits = get_significant_digits(val, n=5)
    max_digit = max(digits) if digits else 9
    val = math.copysign(max_digit * max_val, val)

val = round(val, precision)

return val

```

Purpose: Convert potentially arbitrary bytes → a *safe IEEE-754 float* avoiding:

- NaN
- Inf
- Zero
- Subnormal values
- Overflow

Abnormal cases

- Subnormal numbers
 - *Exponent* == 0 but *mantissa* != 0
 - Very tiny numbers (Closer to 0)
 - Underflow, causing inconsistency
- Zero
 - Exponent and mantissa are both 0
 - Need to avoid too many 0s
- Inf/NaN
 - *Exponent* == 255 (All 1s)
 - *Mantissa* == 0 (Infinity)
 - *Mantissa* != 0 (Nan)

Fixes:

- Zero
 - *Exponent* 0 $\rightarrow$ 127
 - *Mantissa* 0 $\rightarrow$ 1
- Subnormal
 - *Exponent* 0 $\rightarrow$ 127
 - *Mantissa* 1 *unchanged*
- Infinity
 - *Exponent* 255 $\rightarrow$ 127
 - *Mantissa* 0 $\rightarrow$ 1
- NaN
 - *Exponent* 255 $\rightarrow$ 127
 - *Mantissa* 0 $\times \dots \rightarrow$ 1

Then clamp the result into safe bounds while keeping data entropy.

```
if len(b) < 4:

    # pad if necessary

    b = b.ljust(4, b'\x00')
```

- IEEE float32 requires **4 bytes**
- Pads short values with zero bytes $\rightarrow$ avoids struct errors

```
bits = int.from_bytes(b, byteorder='big')
```

Interpret raw bytes as unsigned integer $\rightarrow$ 32-bit binary representation

Turning bytes into a float safely

```
sign = (bits >> 31) & 0x1

exponent = (bits >> 23) & 0xFF

mantissa = bits & 0x7FFFFFFF
```


Extract IEEE 754 fields:

sign: 1 bit

exponent: 8 bits

mantissa: 23 bits

Denormal protection

```
orig_exponent = exponent
```

```
if exponent == 0 or exponent == 255:
```

```
    exponent = 127
```

```
# bias for exponent 0 → safe normal number
```

- Exponent 0 or 255 represent:
 - Zero
 - Inf
 - NaN
 - Subnormals → mathematically risky
- Replace with exponent 127 → **normalized safe value around 1.×**

Avoid Zero mantissa

```
orig_mantissa = mantissa
```

```
if mantissa == 0:
```

```
    mantissa = 1
```

Guarantees non-zero significand → no exact zeros → avoids collapse to 0

```
safe_bits = (sign << 31) | (exponent << 23) | mantissa
```

```
safe_bytes = safe_bits.to_bytes(4, byteorder='big')
```

Reassemble as a corrected IEEE float bit pattern.

```
val = struct.unpack('>f', safe_bytes)[0]
```

Convert into a real float32 Python.

Custom value-range clamping

```
min_val = 1e-12

max_val = 1e12

abs_val = abs(val)
```

- Allowed numeric range to prevent extreme float instabilities
- compare magnitude ignoring sign

Small magnitude handling

```
if abs_val < min_val:

    digits = get_significant_digits(val, n=5)

    max_digit = max(digits) if digits else 1

    val = math.copysign(max_digit * min_val, val)
```

- Too-small numbers are boosted using largest significant digit
- Keeps value structure tied to original data, not constant clamp
- Preserves sign via `copysign` which prevents collapsing to positive only

Large magnitude handling

```
elif abs_val > max_val:

    digits = get_significant_digits(val, n=5)

    max_digit = max(digits) if digits else 9

    val = math.copysign(max_digit * max_val, val)
```

- Too-large numbers → scaled down using most impactful digit
- Both of these add:
 - Stability
 - Data dependency
 - Irreversibility without loss of uniqueness
- **This is chaotic behavior attackers can't algebraically invert.**

```
val = round(val, precision)
```

```
return val
```

- Precision default: 12 decimal places
- Prevents floating point explosion of tail randomness

Formal Mathematical Proof of Non-Invertibility

We can define this transformation function as

$$f: \{0,1\}^* \rightarrow \mathbb{R}$$

- The input to the function is a binary string ($\{0,1\}^*$)
- In practice: *password + salt + HMAC bytes + normalization*
- The output is a real number ($\mathbb{R}$)

specifically: clamped, stabilized float

Let's Find an inverse transformation if exists such that:

$$f^{-1}(f(x)) = x \quad \text{for all } x$$

Domain – Range cardinality mismatch

Input Space

Password Input Space in this System

- Password (Unicode text)
- Encoding (bytes)
- Nonlinear transformation layer
- Argon2 hashing

Unicode Support

Python str $\rightarrow$ *.encode()* produces *UTF – 8 bytes*, system already supports:

- ASCII (English letters)
- Sinhala/Tamil/Hindi characters
- Symbols, punctuation

- Emoji
- ANY Unicode character

Every password becomes a variable-length byte sequence before transformation.

“The system accepts passwords as UTF-8 encoded Unicode strings, resulting in a variable-length byte sequence input to the transformation layer. Therefore, the domain of possible passwords is effectively unbounded, and the pipeline supports any valid Unicode character.”

All possible Unicode passwords the system allows.

PasswordSpace =

$$L = \bigcup_{L=1}^{Lmax} (Unicode)^L \quad \text{where } L \text{ is the length of the password}$$

Encode: Unicode^L → {0,1}^{8k}, k ∈ ℕ

where k is the number of bytes after encoding

Password	Characters (L)	UTF-8 bytes (k)
abc	3	3
ÄbÇ	3	5
❤️🌀	2	8

In UTF-8 encoding:

Character type	Bytes
Basic English letters (ASCII)	1 byte
Extended Latin, accented characters	2 bytes
Common languages (Sinhala/Tamil/Hindi)	2–3 bytes
Emoji / rare symbols	4 bytes

Example: If a password is entered with 4 bytes (with minimal possibilities)

| Domain | = 2³² = 4,294,967,296

Output Space

Output is a rounded float with at most 12 decimal places, inside:

$$10^{-12} \leq |f(x)| \leq 10^{12}$$

A double-precision float has $\sim 2^{53}$ discrete values, but our result **restricts** values to a much smaller, **data-dependent bucketed range** due to *clamping + rounding*.

Thus: $|Range| \ll |Domain|$

By the **pigeonhole principle**:

$$\exists x_1 \neq x_2: f(x_1) = f(x_2)$$

- A function mapping a larger finite set into a smaller one **cannot be injective**, thus cannot be inverted.

Information Erasure (Significand / Exponent Repair)

IEEE-754 float has fields:

$$value = (-1)^{sign} \cdot (1.mantissa) \cdot 2^{(exponent-127)}$$

Modifies exponent and mantissa:

- If $exponent \in \{0, 255\} \rightarrow exponent \leftarrow 127$
- If $mantissa = 0 \rightarrow mantissa \leftarrow 1$

That creates equivalence classes:

$$\forall e \in \{0, 255\}, \forall m = 0: f(s, e, m) = f(s, 127, 1)$$

Infinite distinct byte patterns collapse into 1 output.

Information destroyed: inverse function impossible.

Non-Linear Clamping with Branching

Define:

$$g(x) = \begin{cases} \text{sign}(x) \cdot k_{\min} \cdot 10^{-12} & |x| < 10^{-12} \\ x & 10^{-12} < |x| < 10^{12} \\ \text{sign}(x) \cdot k_{\min} \cdot 10^{12} & |x| > 10^{12} \end{cases}$$

Where $k_{\min}, k_{\max} \in \{1, \dots, 9\}$ come from **significant digits**.

$\{x: |x| < 10^{-12}\} \rightarrow 9 \text{ possible outputs}$

$\{x: |x| > 10^{12}\} \rightarrow 9 \text{ possible outputs}$

Both domains are continuous and uncountably infinite.

Many inputs $\rightarrow$ same output bucket $\rightarrow$ no unique inverse exists.

Concept: Range Buckets;

Compress infinitely many possible input values into **just 18 buckets**:

Case 1 – Very small inputs

$$|x| < 10^{-12}$$

Outputs only,

$$\{\pm(1 \cdot 10^{-12}), \pm(2 \cdot 10^{-12}), \dots, \pm(9 \cdot 10^{-12})\} = 18 \text{ discrete values}$$

But input domain? $\rightarrow$ Infinite. Continuous. Uncountable:

Every real number in the interval $(-10^{-12}, +10^{12})$ maps to only one of 18 outputs.

Case 2 – Huge inputs

$$|x| > 10^{12}$$

Outputs

$$\{\pm(1 \cdot 10^{-12}), \pm(2 \cdot 10^{-12}), \dots, \pm(9 \cdot 10^{-12})\} = 18 \text{ discrete values}$$

Again, Entire **uncountably infinite** real ranges collapse into the same small set of outputs.

To have an inverse, a function must be **injective**:

Every output must come from *one unique* input.

But here:

$g(x_1) = g(x_2)$ for infinitely many $x_1 \neq x_2$

Example:

$$x = 3.1221 \times 10^{-50} \rightarrow g(x) = 3 \times 10^{-12}$$

$$x = 3.3321 \times 10^{-100} \rightarrow g(x) = 3 \times 10^{-12}$$

These are different inputs mapping to **identical** output.

Not injective $\Rightarrow$ No inverse exists

Rounding removes entropy

$$h(x) = \text{round}(x, 12)$$

- This means: keep **12 decimal places**, discard anything smaller.
- Any number in an interval of width 10^{-12} collapses to **the same rounded value**.

Formally:

$$x_1, x_2 \in [a, a + 10^{-12}) \Rightarrow h(x_1) = h(x_2) = a_{\text{rounded}}$$

- Input interval: uncountably infinite real numbers
- Output: **single floating-point number**
- Result: huge **information loss**, i.e., **entropy loss**.

This is exactly what you want **for irreversibility** attackers cannot distinguish x_1 from x_2 after rounding.

Entropy Loss?

Why this does not impact final output security

- **Final password output is still secure because:**
 - The transformation layer already **maps multiple inputs $\rightarrow$ same bucket** (clamping + significant digits)
 - **Rounding only adds more collisions**, increasing **many-to-one mapping**
- **Argon2 hashing comes after rounding:**
 - The output of **round()** is input to Argon2
 - Argon2 is **cryptographically secure**, memory-hard, and one-way
 - Even if rounding reduces “numeric entropy”, Argon2 adds **full cryptographic entropy** to the final hash

- **Numeric precision is already overkill:**
 - Floating-point values in Python (double precision) have ~15–16 decimal digits
 - We round to 12 decimal places, still well above typical collision risk for passwords
 - Any entropy lost here is negligible for cryptographic purposes

3. `def safe_float_from_bytes(b: bytes, precision: int = 12) -> float:`

```
def generate_user_range(salt1: bytes, salt2: bytes) -> Tuple[float, float]:

    def extract_strong_float(hmac_bytes: bytes) -> float:
        val = safe_float_from_bytes(hmac_bytes[:4], precision=6)
        abs_val = abs(val) if val != 0 else 1.0
        exponent = math.log10(abs_val)
        scaled = exponent % 3.0
        res = 10 ** scaled
        return res

    hmac_min = hmac.new(SECRET_KEY, salt1 + salt2 + b'unpredictable_x_min', sha256).digest()
    hmac_max = hmac.new(SECRET_KEY, salt1 + salt2 + b'unpredictable_x_max', sha256).digest()

    min_val = extract_strong_float(hmac_min)
    max_val = extract_strong_float(hmac_max)

    if max_val < min_val:
        min_val, max_val = max_val, min_val

    if abs(max_val - min_val) < 1.0:
        separation_hmac = hmac.new(SECRET_KEY, salt1 + salt2 + b'separation', sha256).digest()
        separation = round(safe_float_from_bytes(separation_hmac[:4]) % 10.0, 6)
        max_val = round(min_val + separation + 1.5, 6)

    return round(min_val, 6), round(max_val, 6)
```

Purpose: produces a *user-specific range* [*min_val*, *max_val*] that will later be used to normalize x-values. It ensures that the transformation adapts to each user.

- Inputs: *salt1* and *salt2* (user-specific randomness / salts)
- Output: tuple (*min_val*, *max_val*) used for normalizing x-values.

`def extract_strong_float(hmac_bytes: bytes) -> float:`


```

val = safe_float_from_bytes(hmac_bytes[:4], precision=6)

abs_val = abs(val) if val != 0 else 1.0

exponent = math.log10(abs_val)

scaled = exponent % 3.0

res = 10 ** scaled

return res

```

Purpose: convert HMAC bytes to a **stable, user-specific float** with variability but safe magnitude.

- Takes the first 4 bytes of HMAC then converts to **safe float** using *safe_float_from_bytes*.
- Takes *abs()* and ensures it's nonzero.
- Computes *log10* to get the order of magnitude.
- Takes *mod 3* → ensures the exponent cycles every 3 orders of magnitude.
- Returns $10 ** scaled$ → produces a float roughly in [1, 1000) range.

Ex: *val* = 0.00345; This is exactly what *safe_float_from_bytes(hmac_bytes[:4])* might return.

```
abs_val = abs(val) = abs(0.00345) = 0.00345
```

```
0.00345 = 3.45 × 10-3 → log10 = log10(3.45) - 3 = 0.5376 - 3 = -2.4624
```

```
scaled = (-2.462398) % 3.0
```

```
-2.462398 + 3 = 0.537602
```

```
So scaled = 0.537602
```

```
res = 10 ** 0.537602
```

```
100.537602 ≈ 3.45
```

The output is the mantissa of the original number, stripped of its exponent

1. Every *float* = *Sign* × *Mantissa* × 2^{*Exponent*}

In IEEE 754 binary32, a float is stored as three parts:

Part	Meaning
Sign bit (S)	+ or –
Exponent (E)	Determines magnitude (scale)
Mantissa (M)	The precise fractional pattern that defines the “shape” of the number

Every number is represented as:

$$Value = (-1)^S \times (1.\text{mantissa bits}) \times 2^{(E-127)}$$

The mantissa always represents a number between **1.0 and 2.0**.

2. What you do when you strip the exponent

When “strip the exponent,” we do this:

- **Keep** the sign bit
- **Keep** the 23 mantissa bits
- **Throw away** the exponent
- **Replace exponent with the neutral one** (*bias 127 → means 2^0*)

This forces the number into the form:

$$New Value = (-1)^S \times (1.\text{mantissa bits}) \times 2^0$$

But:

$$2^0 = 1$$

So the exponent **contributes nothing**.

This leaves only:

$$(-1)^S \times (1.\text{mantissa bits})$$

This is what we mean by:

- Keeping only the mantissa.
- The exponent is removed.

3. Why this drastically changes the number

Let's take a real example:

Original number:

$$x = 13.75$$

IEEE 754 decomposition:

- $Sign = 0$
- $Exponent = 10000010_2 \rightarrow 130 \rightarrow actual\ exponent = 130 - 127 = 3$
- $Mantissa\ bits = 101110000000000000000000$

So:

$$x = 1.10111_2 \times 2^3$$

Now:

When we strip the exponent:

We make the $exponent = 127 \rightarrow actual\ exponent = 0$.

So new value:

$$x' = 1.10111_2 \times 2^0$$

$$x' = 1.6875$$

The magnitude (13.75) disappears,
only the mantissa pattern remains.

4. The mantissa is the precision fingerprint of the number

The mantissa contains all the “**shape**” information, not the scale.

For example:

- 13.75

- 1.71875
- 0.107421875

All can share similar mantissas but with different exponents.

When you strip the exponent: All numbers collapse into the range:

$$1.0 \leq \text{value} < 2.0$$

Because the leading bit is always 1 for normalized floats.

This means:

- We removed magnitude information
- We preserved the precision fingerprint

This is the exact meaning of:

Output the mantissa stripped of its exponent.

5. Why the value becomes small and predictable

Since the mantissa is always between 1 and 2, you will always get:

Something between **1.000000** and **1.999999**

- Never negative unless sign bit was negative
- Never huge
- Never tiny

Because the exponent is the part that normally makes numbers huge or tiny.

6. Why this is useful in your hashing method

When you remove the exponent:

- We destroy all predictable magnitude information
 - (attackers cannot approximate size)
- We flatten the domain
 - (all values map into the same small range)
- But keep the fine-grained precision

- (the mantissa pattern is very sensitive to input, increasing unpredictability)
- We avoid overflow/underflow
 - (because exponent is what causes extreme values)
- This is why the output is stable, safe, and high-entropy.

The clearest simple explanation

Imagine a float is made of:

- **WHERE** the number is (*exponent*)
- **WHAT SHAPE** it has (*mantissa*)

When we remove the exponent, we keep only the shape while collapsing its scale.

So we are essentially saying:

“Forget how big the number is, only remember the detailed pattern inside it.”

That detailed pattern is the *mantissa*.

Generate HMACs for min and max

```
hmac_min = hmac.new(SECRET_KEY, salt1 + salt2 + b'unpredictable_x_min',
sha256).digest()
```

```
hmac_max = hmac.new(SECRET_KEY, salt1 + salt2 + b'unpredictable_x_max',
sha256).digest()
```

- Use *HMAC* with secret key and salts to produce deterministic but secret outputs
- *b'unpredictable_x_min' / _max* ensures **different HMACs for min and max**

Extract floats

```
min_val = extract_strong_float(hmac_min)
```

```
max_val = extract_strong_float(hmac_max)
```

Converts the HMAC bytes into two floats using the inner function.

Swap if out-of-order

```
if max_val < min_val:
```

```
min_val, max_val = max_val, min_val
```

Ensures *min_val* <= *max_val*

Ensure separation

```
if abs(max_val - min_val) < 1.0:
```

```
    separation_hmac = hmac.new(SECRET_KEY, salt1 + salt2 + b'separation',
    sha256).digest()
```

```
    separation = round(safe_float_from_bytes(separation_hmac[:4]) % 10.0, 6)
```

```
    max_val = round(min_val + separation + 1.5, 6)
```

- If the min and max are too close → generate **random separation** from another *HMAC*
- Guarantees range has *width* $\geq \sim 1.5$ to 11.5

Round and return

```
return round(min_val, 6), round(max_val, 6)
```

Round to 6 decimal places for stability

Summary: *generate_user_range* gives a **user-specific numeric range** [*min*, *max*] that is deterministic from salts but unpredictable without the secret key. This range is later used to normalize x-values.

4. normalize_multi_dim_x_values_to_custom_range

```
def normalize_multi_dim_x_values_to_custom_range(xs_list: list[list[float]], x_min: float, x_max: float, method: str =
"linear") -> list[list[float]]:
    normalized_all = []
    for idx, vec in enumerate(xs_list):
        arr = np.array(vec, dtype=np.float64)
        if arr.size == 0:
            normalized_all.append([])
            continue
        # Apply chosen normalization
        if method == "linear":
            raw_min, raw_max = np.min(arr), np.max(arr)
            if abs(raw_max - raw_min) < 1e-9:
```

```

        normed = np.full_like(arr, 0.5)
    else:
        normed = (arr - raw_min) / (raw_max - raw_min)
    elif method == "log":
        normed = np.log1p(np.abs(arr))
        norm_min, norm_max = np.min(normed), np.max(normed)
        if abs(norm_max - norm_min) < 1e-9:
            normed = np.full_like(normed, 0.5)
        else:
            normed = (normed - norm_min) / (norm_max - norm_min)
    elif method == "clipped":
        # clip 1st and 99th percentile
        low, high = np.percentile(arr, [1, 99])
        clipped = np.clip(arr, low, high)
        normed = (clipped - low) / (high - low) if abs(high - low) > 1e-9 else np.full_like(arr, 0.5)
    elif method == "tanh":
        # scale into -1..1 using tanh, then map to 0..1
        normed = np.tanh(arr)
        normed = (normed + 1) / 2
    else:
        raise ValueError(f"Unknown normalization method: {method}")
    # Map to [x_min, x_max]
    mapped = x_min + normed * (x_max - x_min)
    mapped_rounded = np.round(mapped, 6).tolist()
    normalized_all.append(mapped_rounded)

return normalized_all

```

- Inputs:
 - *xs_list*: list of vectors of floats (multi-dimensional x-values)
 - *x_min, x_max*: the target range (from *generate_user_range*)
 - *method*: normalization type (*linear, log, clipped, tanh*)
- Output: normalized x-values **mapped to *[x_min, x_max]***

Loop over each vector

```
for idx, vec in enumerate(xs_list):
```

```
    arr = np.array(vec, dtype=np.float64)
```

```
    if arr.size == 0:
```

```
        normalized_all.append([])
```

```
        continue
```

- Converts each list to a NumPy array
- Skip empty vectors

Normalization options

Linear

```
raw_min, raw_max = np.min(arr), np.max(arr)

if abs(raw_max - raw_min) < 1e-9:

    normed = np.full_like(arr, 0.5)

else:

    normed = (arr - raw_min) / (raw_max - raw_min)
```

- Map $[raw_min, raw_max] \rightarrow [0,1]$ linearly
- If all values are identical $\rightarrow$ set to 0.5

Log

```
normed = np.log1p(np.abs(arr))

norm_min, norm_max = np.min(normed), np.max(normed)

...
```

- Take $\log(1 + |x|)$ $\rightarrow$ compress large values, expand small ones
- Then normalize to $[0,1]$

Clipped

```
low, high = np.percentile(arr, [1, 99])

clipped = np.clip(arr, low, high)

normed = (clipped - low) / (high - low) if abs(high - low) > 1e-9 else
np.full_like(arr, 0.5)
```

- Clips extreme values (1st/99th percentile) $\rightarrow$ reduces outlier impact

- Maps clipped values $\rightarrow [0,1]$

Tanh

```
normed = np.tanh(arr)
```

```
normed = (normed + 1) / 2
```

- Maps input $\rightarrow [-1,1] \rightarrow$ then $[0,1]$
- Smooth squashing of very large numbers

Map to $[x_{min}, x_{max}]$

```
mapped = x_min + normed * (x_max - x_min)
```

```
mapped_rounded = np.round(mapped, 6).tolist()
```

```
normalized_all.append(mapped_rounded)
```

- After normalization $\rightarrow$ scale linearly to $[x_{min}, x_{max}]$
- Round to 6 decimal places
- Append to output

Purpose: ensures that each user gets a **personalized, safe range** of x-values, with multi-dimensional support.

5. generate_user_param_range

```
def generate_user_param_range(salt1: bytes, salt2: bytes, label: bytes) -> Tuple[float, float]:

    def extract_strong_float(hmac_bytes: bytes) -> float:
        base_val = safe_float_from_bytes(hmac_bytes[:4], precision=6)
        extra_bits = int.from_bytes(hmac_bytes[4:8], byteorder='big')
        frac_rand = (extra_bits % 10000) / 10000.0
        combined = abs(base_val) * (1 + frac_rand)
        scaled = (math.log10(combined + 1e-12) + 6) % 3.0
        res = round(10 ** scaled, 6)
        return res
```

```

hmac_min = hmac.new(SECRET_KEY, salt1 + salt2 + label + b'_min', sha256).digest()
hmac_max = hmac.new(SECRET_KEY, salt1 + salt2 + label + b'_max', sha256).digest()

min_val = extract_strong_float(hmac_min)
max_val = extract_strong_float(hmac_max)

if max_val < min_val:
    min_val, max_val = max_val, min_val

if abs(max_val - min_val) < 5.0:
    sep_hmac = hmac.new(SECRET_KEY, salt1 + salt2 + label + b'_sep', sha256).digest()
    separation = (int.from_bytes(sep_hmac[:2], 'big') % 50) / 10.0
    max_val = round(min_val + 5.0 + separation, 6)

return round(min_val, 6), round(max_val, 6)

```

Similar to *generate_user_range*, but for individual parameters (like p_1, p_2).

- Uses *extract_strong_float* with **extra fractional randomness** from later *HMAC* bytes (*extra_bits*)
- Adds checks to ensure *min* < *max*
- If range too small, adds random separation (+5.0 + *separation*)
- Rounds final values

Purpose: produce **user-specific parameter ranges** for p_1/p_2 , ensuring each user has unique normalization boundaries.

6. normalize_param

```

def normalize_param(value: float, param_min: float, param_max: float, method: str = "linear") -> float:
    x = np.array([value], dtype=np.float64)
    def linear_norm(x):
        scaled = abs(x) % 1.0
        return param_min + scaled * (param_max - param_min)
    def log_norm(x):
        safe_val = np.clip(np.abs(x), 1e-30, 1e30)
        log_val = np.log10(safe_val)
        log_min, log_max = log_val, log_val
        if abs(log_max - log_min) < 1e-6:
            log_max = log_min + 1.0
        normed = (log_val - log_min) / (log_max - log_min)
        return param_min + normed * (param_max - param_min)

```

```

def clipped_norm(x):
    lower, upper = -1.0, 1.0
    clipped = np.clip(x, lower, upper)
    normed = (clipped - lower) / (upper - lower)
    return param_min + normed * (param_max - param_min)

def tanh_norm(x):
    scale = 1e10
    squashed = np.tanh(x / scale)
    tanh_min, tanh_max = squashed, squashed
    if abs(tanh_max - tanh_min) < 1e-6:
        tanh_max = tanh_min + 1.0
    normed = (squashed - tanh_min) / (tanh_max - tanh_min)
    return param_min + normed * (param_max - param_min)

methods = {
    "linear": linear_norm,
    "log": log_norm,
    "clipped": clipped_norm,
    "tanh": tanh_norm
}

if method not in methods:
    raise ValueError(f"Unknown normalization method: {method}")

normalized = methods[method](x)[0]
return round(normalized, 6)

```

- Normalize a **single scalar** *value* to $[param_min, param_max]$
- Supports *linear, log, clipped, tanh*
- Uses tiny hacks to avoid division by zero / flat arrays

Linear

```
scaled = abs(x) % 1.0
```

```
return param_min + scaled * (param_max - param_min)
```

- Maps any float to $[0,1]$ fraction, then $\rightarrow [param_min, param_max]$

Log

```
safe_val = np.clip(np.abs(x), 1e-30, 1e30)
```

```
log_val = np.log10(safe_val)
```

- Safely compute *log10* of x

- Avoid underflow/overflow
- Map $\rightarrow [param_min, param_max]$

Clipped

`lower, upper = -1.0, 1.0`

`clipped = np.clip(x, lower, upper)`

- Clip extreme values $\rightarrow$ map to 0..1 $\rightarrow$ then param range

Tanh

`scale = 1e10`

`squashed = np.tanh(x / scale)`

`normed = (squashed - tanh_min) / (tanh_max - tanh_min)`

- Smoothly squash extremely large values $\rightarrow$ map to 0..1 $\rightarrow$ param range
- Purpose: allows **fine control** over individual parameters (p_1, p_2 , etc.) while staying in safe numeric ranges

Security Notes

- All normalization is user-specific:
 - Ranges depend on `salt1`, `salt2`, and secret key
 - Makes each user's transformed x-values unique
- All methods protect against extreme floats:
 - Linear/log/tanh/clipped prevent overflows, underflows, NaNs
- Normalization $\rightarrow$ final numeric stability:
 - After this, ***map_char_to_function_with_x*** pipeline can safely use p_1, p_2, x_i values for function evaluation
 - Rounding / clamping and normalization together ensure irreversibility + numerical safety

7. apply_plot_function

```
def apply_plot_function(plot_type: int, p1: float, p2: float, x_list: List[float]) -> Tuple[float, str]:
    n = len(x_list)
    expr_val = 0.0
    expr_str_parts = []
    if plot_type == 0:
        for i in range(n):
            for j in range(n):
                term_val = ((-1)**(i+j)) * p1 * x_list[i] * x_list[j] - p2 * x_list[i]**(j+2)
                expr_val += term_val
                expr_str_parts.append(f"((-1)**(i+j))*{p1}*x{i+1}*x{j+1} - {p2}*x{i+1}^{j+2}")
    elif plot_type == 1:
        for i in range(n):
            term_val = p1 * math.sin(x_list[i]) + p2 * x_list[i]**3
            expr_val += term_val
            expr_str_parts.append(f"{p1}*sin(x{i+1}) + {p2}*x{i+1}^3")
    elif plot_type == 2:
        for i in range(n):
            term_val = p1 * math.log(1 + abs(x_list[i])) + p2 * x_list[i]**2
            expr_val += term_val
            expr_str_parts.append(f"{p1}*log(1+|x{i+1}|) + {p2}*x{i+1}^2")
    elif plot_type == 3:
        for i in range(n):
            term_val = p1 * math.exp(x_list[i]) - p2 * x_list[i]**2
            expr_val += term_val
            expr_str_parts.append(f"{p1}*exp(x{i+1}) - {p2}*x{i+1}^2")
    elif plot_type == 4:
        for i in range(n):
            term_val = p1 * x_list[i]/(1 + x_list[i]**2) - p2 * x_list[i]**3
            expr_val += term_val
            expr_str_parts.append(f"{p1}*x{i+1}/(1+x{i+1}^2) - {p2}*x{i+1}^3")
    elif plot_type == 5:
        for i in range(n):
            for j in range(n):
                term_val = ((-1)**i)*p1*math.sin(x_list[i]) + p2*math.exp(x_list[j]) - math.log(1 +
abs(x_list[i]*x_list[j]))
                expr_val += term_val
                expr_str_parts.append(f"((-1)**i)*{p1}*sin(x{i+1}) + {p2}*exp(x{j+1}) - log(1+|x{i+1}*x{j+1}|)")
    func_repr = " + ".join(expr_str_parts)
    return expr_val, func_repr
```

What the function does (high level)

`apply_plot_function(plot_type, p1, p2, x_list):`

- Takes scalar parameters $p1, p2$ and a list of n float coordinates $x_1 \dots x_n$.
- Computes a single scalar $expr_val$ that is the sum of many nonlinear terms depending on $plot_type$.
- Builds a human-readable string $func_repr$ showing the symbolic expression that was summed.
- Returns $(expr_val, func_repr)$.

Complexity:

- $plot_type$ 0 & 5: double loops $\rightarrow O(n^2)$ arithmetic operations.
- Types 1, 2, 3, 4: single loop $\rightarrow O(n)$.

$n = len(x_list)$ and x_i in math notation correspond to $x_list[i]$ (the code prints $x\{i + 1\}$).

Mathematical representations

- Let $n = \#(x_list)$ and x_i be the $i - th$ element of x_list .
- Let $X = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$ be the vector of transformed coordinates (these are produced from $HMAC \rightarrow safe_float \rightarrow mantissa/clamp/normalize$).
- Let $F_k(X)$ be the scalar produced by $plot_type = k$.
- We show $F_k: \mathbb{R}^n \rightarrow \mathbb{R}$ is non-injective and that solving $F_k(X) = y$ yields large preimage sets (manifolds / infinite families).

Formal tool: level sets and dimensions

For any continuous differentiable $F_k: \mathbb{R}^n \rightarrow \mathbb{R}$, a level set

$$L_y = \{X \in \mathbb{R}^n: F(X) = y\}$$

Is (when regular) an $(n - 1)$ - dimensional manifold (curve for $n = 2$, surface for $n = 3$, etc.). So:

If $n \geq 2$, L_y is generically infinite (uncountable).

Thus F cannot be injective for $n \geq 2$ (many X map to the same y).

This is the core topological reason this mapping is irreversible: we compress n degrees of freedom to 1; the pre-image generically has $n - 1$ degrees of freedom.

plot_type = 0

for i in range(n):

for j in range(n):

term = ((-1)**(i+j)) * p1 * x_i * x_j - p2 * x_i**(j+2)

expr_val += term

$$F_0(p_1, p_2, x) = \sum_{i=1}^n \sum_{j=1}^n [(-1)^{i+j} p_1 x_i x_j - p_2 x_i^{j+2}]$$

First part: a bilinear, signed interaction $p_1 \sum_{ij} (-1)^{i+j} p_1 x_i x_j$

Second part: highly non-linear powers $p_2 x_i^{j+2}$ note the exponent depends on j , so exponents range from 3 to $n + 2$).

Why non-injective (intuitively)

- The bilinear block $p_1 \sum_{ij} (-1)^{i+j} p_1 x_i x_j$ is a quadratic form $X^T A X$ for some symmetric matrix A . Quadratic forms map many different vectors to the same scalar (e.g., X and $-X$ give the same for even forms).
- The second term uses high-degree powers $p_2 x_i^{j+2}$ with exponents depending on indices; but because many variables are involved, you have a polynomial equation in the unknowns: $\sum_{ij} P_{ij}(x_i x_j) = y$. For $n \geq 2$ that polynomial equation defines an $(n - 1)$ - dimensional algebraic variety: infinite solutions.

Small concrete example ($n = 2, p_1 = p_2 = 1$):

$$F_0(x_1, x_2) = \sum_{i=1}^2 \sum_{j=1}^2 (-1)^{i+j} x_i x_j - \sum_{i=1}^2 \sum_{j=1}^2 x_i^{j+2}$$

$$F_0 = (+1)x_1 x_1 + (-1)x_1 x_2 + (-1)x_2 x_1 + (+1)x_2 x_2 - (x_1^3 + x_1^4 + x_2^3 + x_2^4)$$

$$F_0 = x_1^2 - 2x_1 x_2 + x_2^2 - (x_1^3 + x_1^4 + x_2^3 + x_2^4)$$

$$F_0 = (x_1 - x_2)^2 - (x_1^3 + x_1^4 + x_2^3 + x_2^4)$$

Let's say for a fixed output $y = 0$ (If somehow reverse engineered the attacker finds y);

$$0 = (x_1 - x_2)^2 - (x_1^3 + x_1^4 + x_2^3 + x_2^4)$$

$$(x_1 - x_2)^2 = (x_1^3 + x_1^4 + x_2^3 + x_2^4)$$

Is a 2-variable polynomial curve which has infinitely many (x_1, x_2) satisfy it. Hence non-invertible.

Security implication: Because the polynomial equation is high-degree and mixed, root-finding is numerically unstable and has multiple branches. Attackers cannot uniquely recover X .

A single equation such as:

$$F(x_1, x_2) = 0$$

Defines a **curve** in the 2-dimensional plane, not a single point. (Which means to find the solution we need both x_1 and x_2)

- If you have **two unknowns** and **one equation**,
The solution is always a **1-dimensional object** (a curve).

A curve always has **infinitely many points**.

This is simply due to dimensionality.

Think of:

$$x_1 + x_2 = 5$$

That has **infinite pairs** (x_1, x_2) .

Same principle is applied to;

$$(x_1 - x_2)^2 - (x_1^3 + x_1^4 + x_2^3 + x_2^4) = c \quad \text{where } c \in \mathbb{R}$$

plot_type = 1

for i in range(n):

term = p1 * sin(x_i) + p2 * x_i**3

$$F_1 = \sum_{i=1}^n p_1 \sin(x_i) + p_2 x_i^3$$

Why non-injective

$\sin(x)$ is periodic: $\sin(x) = \sin(x + 2\pi k)$ for any integer k . So even for single-component terms there are infinitely many x_i that map to the same $\sin(x_i)$.

Cubic term x^3 is injective on $\mathbb{R}$, but combined with $\sin(x)$ and the sum over multiple coordinates you get many degrees of freedom.

Concrete example ($n = 1$ vs. $n = 2$):

- If $n = 1$: $F_1(x) = p_1 \sin(x) + p_2 x^3$. This is a 1D nonlinear function; theoretically invertible on restricted domains but globally multi-rooted because of the periodic $\sin$. For a value y , equation $p_1 \sin(x) + p_2 x^3 = y$ can have multiple real solutions due to sine oscillation.
- If $n = 2$: $F_1(x_1, x_2) = p_1(\sin(x_1) + \sin(x_2)) + p_2(x_1^3 + x_2^3)$ For fixed y , you can vary x_2 arbitrarily and solve for x_1 (1 DOF – Degree Of Freedom), so infinite pre-images: e.g., pick any a , solve $p_1 \sin(x_1) - p_2 x_1^3 = y - p_1 \sin(a) + p_2 a^3$. Often there is at least one solution x_1 for each a .

Practical attack resistance

- Sine periodicity creates many local *minima/maxima* and multiple roots which makes root finding expensive and ambiguous.
- Because $HMAC \rightarrow x_i$ are secret and normalized, attacker doesn't have an explicit mapping from password candidate to x_i without re-computing $HMAC$ (which requires secret key), so plotting $\sin(x)$ constant does not give attacker the original x_i values.

plot_type = 2

```
for i in range(n):
```

```
    term = p1 * log(1 + |x_i|) + p2 * x_i**2
```

$$F_2 = \sum_{i=1}^n p_1 \log(1 + |x_i|) + p_2 x_i^2$$

Using $\log = \text{natural log}$; $\log(1 + |x|)$ protects domain and is smooth at 0.

Why non-injective

- The x_i^2 term is even: x_i and $-x_i$ give same contribution. So sign is lost so becomes immediately non-injective.

- $|x_i|$ in the log also loses sign, so both terms are invariant under $x_i \mapsto -x_i$.

For $n \geq 1$, each coordinate has sign ambiguity; for $n \geq 2$ combination yields a large set of pre-images (e.g., flipping signs coordinate-wise gives 2^n discrete symmetry even before considering continuous families).

Simple example ($n = 1$):

$$p_1 \log(1 + |x|) + p_2 x^2 = y$$

- If x is a solution, $-x$ is also solution Therefore at least two preimages. For larger n combinatorial multipliers.
- **Security implication:** sign losses and absolute-values create multiple symmetric pre-images, a direct barrier to unique inversion and a source of many-to-one mapping.

plot_type = 3

for i in range(n):

term = p1 * exp(x_i) - p2 * x_i**2

$$F_3 = \sum_{i=1}^n p_1 e^{x_i} - p_2 x_i^2$$

Why non-injective

- e^x is monotone increasing and convex, so per-coordinate it is invertible; however:
 - For $n \geq 2$, you again have an $(n - 1)$ - dimensional level set for fixed y .
 - The combination $p_1 e^{x_i} - p_2 x_i^2$ is non-linear and can have multiple real x_i values producing the same per-coordinate contribution because the downward quadratic term may produce competing effects at different ranges (if coefficients sign/scale appropriate).
- Even if single-variable sub-equations are invertible, the multi-variable sum leaves infinite solutions.
 - Example ($n = 2$): choose x_2 arbitrarily, solve scalar equation in x_1 for the remainder typically solvable because exponential dominates for large x , quadratic for others. So infinite family of solutions.
 - **Numerical inversion hardness:** solving mixed *exp* – *poly* nonlinear equations robustly is computationally expensive and numerically unstable; not feasible for mass brute-force.

plot_type = 4

for i in range(n):

term = p1 * x_i/(1 + x_i**2) - p2 * x_i**3

$$F_4 = \sum_{i=1}^n (p_1 \frac{x_i}{1+x_i^2} - p_2 x_i^3)$$

Why non-injective

The rational term $x/(1+x^2)$ is bounded and odd; cubic term is odd but strongly non-linear. Combined, for fixed y you still get large pre-image families for $n \geq 2$.

The function is non-linear and not monotone globally, producing multiple roots even in $1D$ in some parameter choices.

Example ($n = 2$): again pick one coordinate and solve for the other $\rightarrow$ infinite solutions.

plot_type = 5

for i in range(n):

for j in range(n):

term = ((-1)**i)*p1*sin(x_i) + p2*exp(x_j) - log(1 + abs(x_i*x_j))

$$F_5 = \sum_{i=1}^n \sum_{j=1}^n [(-1)^i p_1 \sin(x_i) + p_2 e^{x_j} - \log(1 + |x_i x_j|)]$$

- This actually repeats some terms many times (the $\sin(x_i)$ term repeated for every j , and e^{x_j} repeated for every i), which multiplies their contribution by n .

Why non-injective & very strong

- This mixes every pair (i, j) : $\sin(x_i)$ repeated across j , $\exp(x_j)$ repeated across i , and $\log(1 + |x_i x_j|)$ coupling pairs multiplicatively.

- The coupling term $\log(1 + |x_i x_j|)$ entangles coordinates multiplicatively, solving for any one coordinate requires global reasoning about all others.
- For $n \geq 2$ the level set is highly nonlinear and intertwined plus extremely hard to invert.
- Concrete small example ($n = 2$): you get a system where each summand depends on both x_1 and x_2 in nontrivial ways; algebraic isolation is effectively impossible without iterating over continuous families.

General mathematical proofs (statements can be used in the paper)

- **Cardinality / dimension argument**
 - If $n \geq 2$, mapping $F: \mathbb{R}^n \rightarrow \mathbb{R}$ compresses n degrees of freedom to 1; generically the pre-image $F^{-1}(y)$ is an $(n - 1)$ - dimensional manifold $\rightarrow$ uncountably infinite. Thus any such F is non-injective.
 - Formal: If $\nabla F(x) \neq 0$ on the level set, the implicit function theorem says the level set is an $(n - 1)$ - dim manifold; else critical points can create even larger degeneracy.
- **Continuum $\rightarrow$ finite/low-bit output (pigeonhole-like)**
 - The pipeline reduces entropy via rounding/clamping; then the plot function aggregates. Combined,
 - $|Domain| \gg |Possible\ numeric\ outputs|$. Hence collisions exist.
- **Transcendental / periodic functions create many roots**
 - Periodicity ($\sin$) and absolute-value symmetries produce countably infinite and discrete symmetries; coupling across coordinates makes preimage structure even more complex.
- **Algebraic hardness**
 - For many of these functions, inversion reduces to solving high-degree polynomial/transcendental equations. There are no general closed-form solvers and numerical root-finding has exponential complexity in required precision and number of solutions.

Why multi-dimensional unpredictable functions are better than single-dimensional $f(x)$

Single-dim $f(x)$ vulnerabilities

If you used a single scalar function $g(x)$ where x is derived deterministically from the password (and all parties see the function), an attacker could:

Plot g ,

For a given output y find all x with $g(x) = y$,

If the mapping $password \rightarrow x$ is simple/known (e.g., $x = \text{some fixed encoding}(password)$), map those x back to small password candidates.

**** Even if g is complicated, plotting y vs. x is feasible and root-finding in 1D is cheap and well-understood; for many functions you can get closed-form inverses or accurate numerical root-finding quickly. ****

Why multi-dimensional approach is safer

- **High-dim pre-image:** compressing $\mathbb{R}^n \rightarrow \mathbb{R}$ means each y corresponds to a manifold of pre-images attacker must search an uncountable set or discretize it heavily.
- **Coupling terms** (like $\log(1 + |x_i x_j|)$) create multiplicative entanglement between coordinates cannot reduce to independent 1D problems.
- **Periodic / non-monotone components** add multiple local solutions and oscillations: root-finding explodes combinatorially with dimension.
- **Secret-driven inputs** ($HMAC + per - user param ranges$) mean attacker cannot even generate the correct $x - space$ mapping without the *secret key*.
- **Cost amplification:** even if attacker reduces candidate set for x , mapping back to password requires re-computing expensive *HMACs and Argon2 per candidate* $\rightarrow expensive$.

Practical, provable security claims (to include in the paper)

Theorem (Informal): For $n \geq 2$, any smooth $F: \mathbb{R}^n \rightarrow \mathbb{R}$ that mixes coordinates nontrivially has infinite preimage sets for all regular output values y . Therefore it is not invertible.

Proof sketch: implicit function theorem / level-set $dimension = n - 1$.

Corollary: The attacker cannot produce a deterministic inverse F^{-1} mapping a scalar y back to a unique x .

Practical security statement: Even if attacker knows F (the algebraic form), recovering original password requires:

(i) Solving for a continuum of possible x (numerically hard),

(ii) For each candidate x finding a password preimage of the secret $HMAC \rightarrow float\ mapping$ (infeasible without secret key), and

(iii) Verifying with Argon2 (expensive). Thus end-to-end cost per password guess is multiplied by multiple expensive operations.

8. def map_char_to_function_with_x

```
def map_char_to_function_with_x(char: str, salt1: bytes, salt2: bytes, char_index: int, max_dims: int = 8) -> Tuple[int, float, float, int, List[float]]:

    # plot type
    tag = char.encode('utf-8') + bytes([char_index]) # use i as unique per character instance
    hmac_type = hmac.new(SECRET_KEY, salt1 + salt2 + tag + b'type', sha256).digest()
    plot_type = hmac_type[0] % 6
    print(" plot_type -> {}", plot_type)

    # p1 and p2 base values
    hmac_p1 = hmac.new(SECRET_KEY, salt1 + salt2 + tag + b'p1', sha256).digest()
    raw_p1 = safe_float_from_bytes(hmac_p1[:4])

    hmac_p2 = hmac.new(SECRET_KEY, salt1 + salt2 + tag + b'p2', sha256).digest()
    raw_p2 = safe_float_from_bytes(hmac_p2[:4])

    # normalize p1 and p2 into user-specific ranges
    p1_min, p1_max = generate_user_param_range(salt1, salt2, b'p1')
    p2_min, p2_max = generate_user_param_range(salt1, salt2, b'p2')
    p1 = normalize_param(raw_p1, p1_min, p1_max, method="linear")
    p2 = normalize_param(raw_p2, p2_min, p2_max, method="linear")

    print("raw_p1 before normalization:", raw_p1)
    print("p1 after normalization:", p1)
    print("raw_p2 before normalization:", raw_p2)
    print("p2 after normalization:", p2)

    # determine n (dimensions) via a dedicated HMAC
    hmac_n = hmac.new(SECRET_KEY, salt1 + salt2 + tag + b'n_dims', sha256).digest()
    n = (hmac_n[0] % max_dims) + 1

    # For x_i values, derive using successive HMACs/blocks so each xi is independent
    x_list: List[float] = []
    for i in range(n):
        block_tag = b'x' + bytes([i])
        hmac_xi = hmac.new(SECRET_KEY, salt1 + salt2 + tag + block_tag, sha256).digest()
        xi = safe_float_from_bytes(hmac_xi[:4])
        x_list.append(xi)
```

```
print(" final mapping: plot_type={}, p1={}, p2={}, n={}, x_list={}", plot_type, p1, p2, n, x_list)
return plot_type, p1, p2, n, x_list
```

High-level Purpose

For every character of the password, this function generates a completely unique multivariate mathematical function, with:

- a plot type (0–5)
- parameters p_1, p_2
- a dimension count n
- a list of $x_1, x_2, \dots, x_n$ values

All of these are deterministically random, user-specific, salt-dependent, and character-instance–dependent.

This means:

Even the same character “a” appearing twice produces *different* functions because *char_index* is included.

Generate plot type using HMAC

```
tag = char.encode('utf-8') + bytes([char_index])
```

```
hmac_type = hmac.new(SECRET_KEY, salt1 + salt2 + tag + b'type',
sha256).digest()
```

```
plot_type = hmac_type[0] % 6
```

What is happening?

- $tag = (char + position\ index)$
- We compute:

$$HMAC(SECRET_KEY, salt1 || salt2 || tag || "type")$$

- $hmac_type[0] = \text{first byte of HMAC} \rightarrow \text{value } 0\text{--}255$

- $plot_type = byte \% 6 \rightarrow$ gives a number in 0–5

Why this is secure

- HMAC makes the output **unpredictable**.
- Using both salts ensures **user-specific randomness**.
- Adding *char_index* ensures two identical characters map differently.
- No attacker can guess the plot type without the HMAC key.

Generate raw p1 and p2 values

```
hmac_p1 = hmac.new(... b'p1').digest()
raw_p1 = safe_float_from_bytes(hmac_p1[:4])
hmac_p2 = hmac.new(... b'p2').digest()
raw_p2 = safe_float_from_bytes(hmac_p2[:4])
```

What is happening?

- Two more independent *HMACs*.
- Only the first 4 bytes are used $\rightarrow$ treated as *IEEE754 float32*.

Security reason

- Each character gets **two private parameters**, unpredictable but reproducible.
- Even the **smallest change** in input, salt, or index $\rightarrow$ completely different p_1 and p_2 .

Normalize p1 and p2 into user-specific ranges

```
p1_min, p1_max = generate_user_param_range(... b'p1')
p2_min, p2_max = generate_user_param_range(... b'p2')
p1 = normalize_param(raw_p1, p1_min, p1_max)
p2 = normalize_param(raw_p2, p2_min, p2_max)
```


What this does

Each user has a **private** range for p_1 and p_2 , like Example:

User 1: $p_1 \in [3.1, 6.9]$ AND $p_2 \in [-4.1, 5.9]$

User 2: $p_1 \in [-1.2, 0.5]$ AND $p_2 \in [-3.8, 5.9]$

User 3: $p_1 \in [10, 20]$ AND $p_2 \in [30.1, 62.9]$ etc.

p_1/p_2 ranges $\rightarrow$ CHARACTER-SPECIFIC

(Min/Max tied to the character's plot function)

Then you map the HMAC-derived float into that range.

Why this is extremely strong

- No universal values exist across users.
- Even if attacker learns formulas, parameters differ per user.
- Even if same password appears for two users $\rightarrow$ completely different output.

But

Even though all values (p_1/p_2) stay inside the character's own range, this **does not mean** the values are globally unique across all characters.

Example:

'a' $\rightarrow$ p1 range = [1.2, 4.8]

'c' $\rightarrow$ p1 range = [1.0, 5.5]

Ranges can overlap.

But the **noise, normalization, unique seeds, and character-specific functions** make the actual *pattern* of values unique, even if ranges overlap.

So uniqueness comes from the **behavior**, not just the *min/max*.

Select dimension count n (1 to `max_dims`)

```
hmac_n = hmac.new(... b'n_dims', sha256).digest()
```

```
n = (hmac_n[0] % max_dims) + 1
```

For each character, **dimension count** n is random but stable.

Example:

Character "a" at index 0:

$n = 5$ for UserA

$n = 2$ for UserB

$n = 8$ for UserC

Why this matters

This is the key to **multi-dimensional unpredictability**.

1D → Invertible or plottable.

nD → Explosion of pre-images, impossible to visualize.

Generate $x_1, x_2, \dots, x_n$ via independent HMACs

```
for i in range(n):
```

```
    block_tag = b'x' + bytes([i])
```

```
    hmac_xi = hmac.new(SECRET_KEY, salt1 + salt2 + tag + block_tag,
sha256).digest()
```

```
    xi = safe_float_from_bytes(hmac_xi[:4])
```

```
    x_list.append(xi)
```

What this means

Each coordinate x_i is derived from its own HMAC:

`HMAC( key, user_salts || char_tag || "x0" )`

`HMAC( key, user_salts || char_tag || "x1" )`

...

Why this is secure

- Every x_i is independent.
- No pattern between x_i values.
- Extremely difficult to guess or exploit.
- Even similar characters generate radically different x vectors.

Final Output

`(plot_type, p1, p2, n, x_list)`

Creates a *character-specific mathematical system*.

9. def entropy

```
def entropy(data: bytes) -> float:
    counts = Counter(data)
    total = len(data)
    # avoid division by zero
    if total == 0:
        return 0.0
    return -sum((count / total) * np.log2(count / total) for count in counts.values())
```

Computes **Shannon entropy** of a byte sequence.

Entropy measures **how unpredictable/random** the data is.

- **Higher entropy** → more random
- **Lower entropy** → more predictable

Count frequency of each byte

```
counts = Counter(data)
```

- *data* is a byte string (0–255 *values*).
- *Counter* creates a dictionary like:

```
{  
  12: 5,  
  255: 3,  
  87: 10,  
  ...  
}
```

- Each key = a byte value
- Each value = number of times that byte appears

Total number of bytes

```
total = len(data)
```

If no data:

```
if total == 0:
```

```
    return 0.0
```

Entropy must be 0 for empty input.

Compute Shannon entropy

```
return -sum((count / total) * np.log2(count / total) for count in  
counts.values())
```

Breakdown:

For each byte value:

$$p = \frac{\text{count}}{\text{total}} \rightarrow \text{Probability of that byte appearing}$$

Compute contribution to entropy: $p \cdot \log_2(p)$

Collectively, entropy formula is:

$$H = - \sum p_i \log_2(p_i)$$

Where each p_i is the probability of a unique byte

Example (intuitive):

- Data: `b"AAAAAA"`
 - Only one character appears
 - $p = 1.0$
 - Entropy = 0.0 bits → **fully predictable**
- Data: random 256-bit hash
 - All bytes appear in unpredictable frequencies
 - Entropy ≈ 7.9 – 8.0 bits → **max randomness**

10. def iterative_plot_transform

```
def iterative_plot_transform(password: str, salt1: bytes, salt2: bytes, iterations: int):
    current_input = password.encode()
    all_intermediates = []
    for it in range(1, iterations + 1):
        combined_input = current_input + salt1 + salt2
        x_raw_values = []
        # Map each byte/char to function and gather raw x vectors
        for idx, b in enumerate(combined_input):
            char = chr(b % 256)
            plot_type, p1, p2, n, x_list = map_char_to_function_with_x(char, salt1, salt2, char_index=idx)
            x_raw_values.append(x_list)
        # Normalize per-char vectors into the user-specific range
        user_x_min, user_x_max = generate_user_range(salt1, salt2)
        x_values_multi = normalize_multi_dim_x_values_to_custom_range(x_raw_values, user_x_min, user_x_max, "log")
        # Show normalized multi-dim vectors
        for idx, vec in enumerate(x_values_multi):
            print("    normalized vector idx={} -> {}".format(idx, vec))
        # Re-normalize into a safe mathematical range for log and exp
        x_values_multi = normalize_multi_dim_x_values_to_custom_range(x_values_multi, -700, 700, "log")
        # Show normalized multi-dim vectors
        for idx, vec in enumerate(x_values_multi):
            print("    Math normalized vector idx={} -> {}".format(idx, vec))
```

```

# Apply plot functions (one scalar result per character)
values = []
for i, b in enumerate(combined_input):
    char = chr(b % 256)

    plot_type, p1, p2, n, _ = map_char_to_function_with_x(char, salt1, salt2, char_index=i)

    x_vec = x_values_multi[i]
    if len(x_vec) < n:
        x_vec = x_vec + [1e-6] * (n - len(x_vec))
    result, func_str = apply_plot_function(plot_type, p1, p2, x_vec)
    # print("    FUNCTION idx={}| char='{}' -> {}".format(i, char, func_str))
    # print("    OUTPUT  idx={}| char='{}' -> {}".format(i, char, result))
    values.append(result)

    # == Avalanche Amplification via Permutation & Hash Cascade ==
hash_accumulator = bytearray(32) # 32 bytes = 256 bits
for v in values:
    # Convert float to string, hash it
    h = hashlib.sha256(str(v).encode()).digest()
    # XOR cascade (global mixing)
    for i in range(32):
        hash_accumulator[i] ^= h[i]
    # Randomized permutation: rotate accumulator based on hash byte
    shift_amount = h[0] % 32
    hash_accumulator = bytearray(
        hash_accumulator[shift_amount:] +
        hash_accumulator[:shift_amount]
    )
# Convert final mixed hash to float values
# Break into 8 x 4-byte chunks → float32 packable
extra_floats = []
for i in range(0, 32, 4):
    chunk = hash_accumulator[i:i+4]
    val = struct.unpack('!f', chunk)[0] # interpret as float
    if math.isnan(val) or math.isinf(val):
        val = 0.0
    extra_floats.append(val)
# Append these global-mix floats to the per-character values
values.extend(extra_floats)
# Convert to binary with debug
try:
    binary_data = struct.pack(f'{len(values)}f', *values)
except struct.error as e:
    # clamp values to safe float range for packing
    safe_vals = []
    for v in values:
        if math.isnan(v) or math.isinf(v):
            safe_vals.append(0.0)
        else:
            # packable float32 range approx [-3.4e38, 3.4e38]; our clamp is much narrower
            safe_vals.append(max(min(v, 1e30), -1e30))
    binary_data = struct.pack(f'{len(safe_vals)}f', *safe_vals)

all_intermediates.append(binary_data)

```

```
# Next iteration input
# current_input = binary_data
current_input = hashlib.sha256(binary_data).digest()
return current_input, all_intermediates, x_values_multi
```

High-Level Purpose of This Function

For a given password + salts:

1. **Map each character to a plot function** (*type, p1, p2, dimensions*).
2. **Generate x-values**, normalize them *twice* (user-specific + safe math-range).
3. **Apply the character's mathematical plot function** to get a scalar result.
4. **Mix all results globally** using *SHA-256 + XOR avalanche + permutation*.
5. *Convert outputs → float32 → pack → hash.*
6. Repeat for many iterations.

So every iteration transforms the password into a high-entropy unpredictable state.

Initial Setup

```
current_input = password.encode()
```

```
all_intermediates = []
```

- Convert password to bytes (e.g. "abc" → b"abc").
- Store intermediate binary blobs for debugging.

Loop over iterations

```
combined_input = current_input + salt1 + salt2
```

Prepare next iteration

This is critical:

```
current_input = hashlib.sha256(binary_data).digest()
```

DO NOT feed raw floats to next iteration.

Only the **digest**, which is:

- fixed size (32 bytes)
- high entropy
- irreversible

This makes each iteration a **cryptographic round**.

$Hashed(previous_output) || salt1 || salt2$

This ensures each round is **salt-dependent**, **previous output dependent** and **non-repeating**.

Generate raw x-values for each character

For each byte b in *combined_input*:

```
char = chr(b % 256)
```

```
plot_type, p1, p2, n, x_list = map_char_to_function_with_x(...)
```

For every *character instance*, we generate:

- ***plot_type*** (0–5)
- **p_1 & p_2** parameters (unique per-character via *HMAC*)
- **n** = number of dimensions (1–*max_dims*)
- **x_list** = raw float values from the character’s HMAC-derived seeds
- **Per-character randomness is isolated and unique.**

Append each x-list:

```
x_raw_values.append(x_list)
```

At this point we have something like:

```
[  
    [ $x_{11}, x_{12}, x_{13}$ ], ← char 0  
    [ $x_{21}, x_{22}$ ], ← char 1  
    [ $x_{31}, x_{32}, x_{33}, x_{34}$ ], ...  
]
```


Normalize raw x-values into user-specific range

```
user_x_min, user_x_max = generate_user_range(salt1, salt2)

x_values_multi = normalize_multi_dim_x_values_to_custom_range(...
"log")
```

This is the **user personalization stage**:

Every user has a **unique allowed range**, ex: $[-30.4, 87.12]$

- All x-values are squeezed into that range using log-scaling. (Can be changed the type of scaling)

This means:

Even if two users have identical passwords, their math-space coordinates differ → no cross-user collisions.

Normalize AGAIN into safe math range

```
x_values_multi =
normalize_multi_dim_x_values_to_custom_range(x_values_multi, -700,
700, "log")
```

This ensures:

- You never feed dangerous values into $\exp()$, $\log()$, etc.
- Prevents overflow/underflow
- Why ± 700 ?
 - $\exp(700)$ is near float64 limit
 - $\exp(800+)$ → overflow
 - $\exp(-800+)$ → underflow

So this step **guarantees mathematical safety**.

Apply the plot functions (core transformation)

For each character:

1. Recompute that character's $\{plot_type, p1, p2, n\}$

2. Take the normalized vector x_{vec}
3. Apply the mathematical function:

```
result, func_str = apply_plot_function(plot_type, p1, p2, x_vec)
```

These functions are:

- polynomials,
- sine/exp mixtures,
- nonlinear transforms,
- globally non-invertible,
- multi-dimensional

Collect all scalar outputs:

```
values.append(result)
```

So you now have:

```
values = [f_char0_output, f_char1_output, ...]
```

This is **character-level chaos**.

Global Avalanche Mixing

Initialize a 32-byte accumulator

(In programming, an **accumulator** is just a variable (or array) that collects or aggregates values over time.)

```
hash_accumulator = bytearray(32)
```

bytearray(32) → Creates a 32-byte array initialized to all zeros:

```
hash_accumulator = [0, 0, 0, ..., 0] # 32 elements
```

Purpose: to “accumulate” information from each character’s transformed value.

For each scalar v :

- Convert $v \rightarrow \text{string} \rightarrow \text{bytes}$
- Hash it with *SHA256*: *SHA256* hash $\rightarrow$ 32-byte digest h
 - **`h = hashlib.sha256(str(v).encode()).digest()`**

- XOR into accumulator: *XOR* each *byte* of *h* with the current *hash_accumulator*:
 - `for i in range(32):`

```

          hash_accumulator[i] ^= h[i]

          hash_accumulator[i] = hash_accumulator[i] XOR h[i]

```
 - $\wedge$ is *XOR* assignment in Python.
 - *XOR* means: bit flips if bits differ: $0 \wedge 1 = 1, 1 \wedge 1 = 0$, etc.
 - Why *XOR*?
 - It combines values without losing information.
 - It's order-sensitive: changing the order of *vs* changes the result.
 - Produces bit-level avalanche: flipping one bit in *v* flips multiple bits in accumulator.
- Rotate accumulator by a shift based on hash byte
 - `shift_amount = h[0] % 32`

```

hash_accumulator = hash_accumulator[shift_amount:] +
hash_accumulator[:shift_amount]

```
 - Step by step:
 - Take the first byte of *h* $\rightarrow h[0]$ (0–255)
 - *Mod 32* $\rightarrow$ *shift_amount* (0–31)
 - Rotate the accumulator:
 - Suppose *shift_amount* = 5
 - $[a_0, a_1, a_2, \dots, a_{31}] \rightarrow [a_5, a_6, \dots, a_{31}, a_0, a_1, a_2, a_3, a_4]$
 - **Purpose:**
 - Makes the XOR result **position-dependent**
 - Even if two characters produce the same *h*, their effect in the accumulator **depends on order**
 - Increases diffusion (avalanche) across bits

Why this creates avalanche

- Each *v* contributes **32 bytes of hash** to a global accumulator.
- XOR ensures **small differences in v** flip multiple bits.
- Rotation ensures **order matters**, and bits get “mixed across positions”.

- After multiple characters, the accumulator is **highly sensitive to every input byte**.

Key properties achieved

- **Bit-level avalanche:** small change $\rightarrow$ massive change
- **Global mixing:** combines all character outputs into one
- **Order-sensitive:** different password arrangements $\rightarrow$ different outputs
- **Irreversibility:** impossible to recover original $\forall$ s from accumulator alone
- **Brute-force resistant:** attacker must consider all combinations

Example:

$acc = [0, 0, 0, 0, \dots, 0]$ # 32 zeros

One scalar value: $v = 3.14$

$h = SHA256("3.14")$

$h = [0xA3, 0x5C, 0x10, 0xFF, 0x02, \dots]$ # 32 bytes total : Assume the result is this

$h = [163, 92, 16, 255, 2, \dots]$: In decimal

$acc[i] = acc[i] \wedge h[i]$

$acc = [0, 0, 0, \dots]$: Initially

$acc = [163, 92, 16, 255, 2, \dots]$ # same as h : After XOR

$shift_amount = h[0] \% 32$: Compute shift amount

$shift_amount = 163 \% 32$

$163 / 32 = 5 * 32 = 160$ remainder 3

$shift_amount = 3$

$[163, 92, 16, 255, 2, 77, 88, 41, \dots]$: The accumulator currently

$acc = acc[3:] + acc[:3]$: Rotate left by 3, Take elements from index 3 onward + first 3

$acc = [255, 2, 77, 88, 41, \dots, 163, 92, 16]$: Result: moved the first 3 bytes to the end.

Convert accumulator → float values

Break the 32 bytes into 8 chunks of 4 bytes:

```
val = struct.unpack('!f', chunk)[0]
```

If NaN/Inf → set to 0.

These “8 extra floats” are appended:

```
values.extend(extra_floats)
```

This adds **global context** to the character-level values.

Global Context? - A set of features derived from the *whole password*, not any single character.

Why need a Global context?

Character-only system (Local context):

If "abc" and "acb" have character-level outputs:

- both contain a, b, c
- the floats per character are the same
- without mixing, final outputs may be similar

With global context:

- You have 8 floats that depend on the **whole password** structure:
- If you change character order → accumulator changes
- If you change any character value → accumulator changes
- If you add/remove characters → accumulator changes
- These 8 floats carry information *not tied to a specific character*, but tied to the **entire password as a single entity**.

Example:

Character-level floats only (local)

Suppose output floats for characters:

$a \rightarrow [1.1, 2.2]$

$b \rightarrow [3.3, 4.4]$

$c \rightarrow [5.5, 6.6]$

Password “abc” gives:

$[1.1, 2.2, 3.3, 4.4, 5.5, 6.6]$

Password “acb” gives:

$[1.1, 2.2, 5.5, 6.6, 3.3, 4.4]$

Same values, just rearranged.

This is weak.

When adding global 8 floats

extra_floats for "abc" $\rightarrow [0.123, -55.2, 88.1, \dots]$

extra_floats for "acb" $\rightarrow [90.33, 11.7, -7.6, \dots]$

Now the final output is:

characters values + global context floats

These 8 floats are completely different even though characters are the same.

So now:

$"abc" \neq "acb"$

$"abc" \neq "abd"$

$"abc" \neq "xbc"$

Even small changes ripple through globally.

Why it is important:

- Adds avalanche effect
- Prevents reordering attacks
- Makes the transform irreversible
- Creates non-local dependencies
- Ensures whole-password uniqueness

Pack all floats into binary

Serialize the floats into a byte-string:

```
binary_data = struct.pack(f'{len(values)}f', *values)
```

If packing error → clamp to safe range ($\pm 1e30$) and retry.

Stores this:

```
all_intermediates.append(binary_data)
```

Final Output (After preparation for the next iteration)

After all iterations:

```
return current_input, all_intermediates, x_values_multi
```

- *current_input* = final hash (32 bytes)
- *all_intermediates* = binary snapshots from every round: For debugging
- *x_values_multi* = final normalized x – vectors: For debugging

In Summary:

- A multi-stage, nonlinear, multi-dimensional, user-specific, salt-specific, non-invertible transformation pipeline that:
 - Uses cryptographic randomness per-character (HMAC)
 - Normalizes into user-specific coordinate spaces
 - Applies math functions that are one-way by design
 - Mixes globally using XOR avalanche & permutation
 - Extracts chaos into float32
 - Feeds only the SHA256 of the result into the next round
 - Repeats for N iterations
- This is essentially:
 - A nonlinear transform layer
 - A chaos generator
 - A user-personalized function space
 - A cryptographic mixing stage
 - A multi-iteration strengthening pipeline

It strongly amplifies bit-differences and destroys structural predictability.

11. def compare_methods

```
def compare_methods(password: str):
    salt = os.urandom(16)
    salt1 = os.urandom(8)
    salt2 = os.urandom(8)
    results = {}

    # SHA256
    tracemalloc.start()
    start = time.time()
    sha = hashlib.sha256(password.encode() + salt).digest()
    sha_time = time.time() - start
    current, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    sha_memory_kb = peak / 1024
    sha_entropy = entropy(sha)
    sha_len = len(sha)
    sha_verify = (hashlib.sha256(password.encode() + salt).digest() == sha)
    results['SHA256'] = {
        'time': sha_time,
        'entropy': sha_entropy,
        'output_len': sha_len,
        'memory': f"{sha_memory_kb:.3f} KB",
        'verified': sha_verify,
        'hash': sha.hex(),
        'salt': salt.hex()
    }

    # bcrypt
    tracemalloc.start()
    start = time.time()
    bcrypt_hash = bcrypt.hashpw(password.encode(), bcrypt.gensalt())
    bcrypt_time = time.time() - start
    current, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    bcrypt_memory_kb = peak / 1024
    bcrypt_entropy = entropy(bcrypt_hash)
    bcrypt_len = len(bcrypt_hash)
    bcrypt_verify = bcrypt.checkpw(password.encode(), bcrypt_hash)
    results['bcrypt'] = {
        'time': bcrypt_time,
        'entropy': bcrypt_entropy,
        'output_len': bcrypt_len,
        'memory': f"{bcrypt_memory_kb:.3f} KB",
        'verified': bcrypt_verify,
        'hash': bcrypt_hash.decode(),
    }
```



```

        'salt': bcrypt_hash[:29].decode()
    }

    # scrypt
    scrypt_n = 2**14
    scrypt_r = 8
    scrypt_p = 1
    tracemalloc.start()
    start = time.time()
    scrypt_hash = hashlib.scrypt(password.encode(), salt=salt, n=scrypt_n, r=scrypt_r, p=scrypt_p)
    scrypt_time = time.time() - start
    current, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    scrypt_memory_kb = peak / 1024
    scrypt_entropy = entropy(scrypt_hash)
    scrypt_len = len(scrypt_hash)
    scrypt_verify = (hashlib.scrypt(password.encode(), salt=salt, n=scrypt_n, r=scrypt_r, p=scrypt_p) == scrypt_hash)

    results['scrypt'] = {
        'time': scrypt_time,
        'entropy': scrypt_entropy,
        'output_len': scrypt_len,
        'memory': f"{scrypt_memory_kb:.3f} KB",
        'verified': scrypt_verify,
        'hash': scrypt_hash.hex(),
        'salt': salt.hex()
    }

    # Argon2
    hasher = PasswordHasher(time_cost=2, memory_cost=102400, parallelism=8)
    tracemalloc.start()
    start = time.time()
    argon_hash = hasher.hash(password)
    argon_time = time.time() - start
    current, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    argon_memory_kb = peak / 1024
    argon_entropy = entropy(argon_hash.encode())
    argon_len = len(argon_hash.encode())
    try:
        argon_verify = hasher.verify(argon_hash, password)
    except Exception as e:
        argon_verify = False
    results['Argon2'] = {
        'time': argon_time,
        'entropy': argon_entropy,
        'output_len': argon_len,
        'memory': f"{argon_memory_kb:.3f} KB",
        'verified': argon_verify,
        'hash': argon_hash,
        'salt': "Embedded in hash string"
    }

    # Iterative Plot-only / Plot+Argon2
    iterations = 2 # you can adjust this
    final_binary, all_intermediates, x_values_multi = iterative_plot_transform(password, salt1, salt2, iterations)

```

```

# Plot-only (final iteration)
tracemalloc.start()
start = time.time()
plot_hash = hashlib.sha256(final_binary).digest()
plot_time = time.time() - start
current, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()
plot_memory_kb = peak / 1024
plot_entropy = entropy(final_binary)
plot_len = len(plot_hash)
plot_verify = (hashlib.sha256(final_binary).digest() == plot_hash)
results['Plot-only'] = {
    'time': plot_time,
    'entropy': plot_entropy,
    'output_len': plot_len,
    'memory': f"{plot_memory_kb:.3f} KB",
    'verified': plot_verify,
    'hash': plot_hash.hex(),
    'salt': f"{salt1.hex()} + {salt2.hex()}",
    'iterations': iterations
}

# Plot+Argon2
tracemalloc.start()
start = time.time()
plot_argon_hash = hasher.hash(final_binary)
plot_argon_time = time.time() - start
current, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()
plot_argon_memory_kb = peak / 1024
plot_argon_entropy = entropy(plot_argon_hash.encode())
plot_argon_len = len(plot_argon_hash.encode())
try:
    plot_argon_verify = hasher.verify(plot_argon_hash, final_binary)
except Exception as e:
    plot_argon_verify = False
results['Plot+Argon2'] = {
    'time': plot_argon_time,
    'entropy': plot_argon_entropy,
    'output_len': plot_argon_len,
    'memory': f"{plot_argon_memory_kb:.3f} KB",
    'verified': plot_argon_verify,
    'hash': plot_argon_hash,
    'salt': f"{salt1.hex()} + {salt2.hex()}"
}

return results, x_values_multi

```

High-Level Purpose

compare_methods(password):

Benchmarks multiple hashing methods

Measures:

- time
- entropy
- output length
- memory usage (live measurement)
- verification correctness

Includes:

- SHA256
- bcrypt
- scrypt
- Argon2
- The **iterative plot transform**
 - Plot-only
 - Plot+Argon2

It returns:

***results** → full statistics for each method*

***x_values_multi** → debugging output for your plot – transform*

Salt generation

```
salt = os.urandom(16)
```

```
salt1 = os.urandom(8)
```

```
salt2 = os.urandom(8)
```

- *salt* → used for SHA256 and scrypt
- *salt1* and *salt2* → used in the plot-transform
- All are cryptographically random

Each hash has unique randomness.

Prevents rainbow tables.

Makes identical passwords produce different hashes.

SHA256 Block

1. Start memory tracking:

```
tracemalloc.start()
```

This enables memory measurement.

2. Measure execution time:

```
start = time.time()
```

```
sha = hashlib.sha256(password.encode() + salt).digest()
```

```
sha_time = time.time() - start
```

SHA256 hashing is usually extremely fast (microseconds).

3. Measure memory peak

```
current, peak = tracemalloc.get_traced_memory()
```

```
tracemalloc.stop()
```

```
sha_memory_kb = peak / 1024
```

Peak memory used during hashing.

4. Entropy

```
sha_entropy = entropy(sha)
```

Entropy of final 32-byte SHA256 digest.

5. Length

```
sha_len = len(sha)
```

Always 32 bytes.

6. Verification

```
sha_verify = (hashlib.sha256(password.encode() + salt).digest() ==
sha)
```

Re-hashing with same salt should match.

bcrypt Block

bcrypt internally:

- Adds its own random salt
- Cost factor = 12 by default
- Slows hashing intentionally

Same measurement steps: time, memory, entropy, length, verify

'bcrypt_hash[:29]' = salt encoded in bcrypt's own format

- bcrypt embeds salt inside the final encoded string.

How and Why?

When we run:

```
bcrypt_hash = bcrypt.hashpw(password.encode(), bcrypt.gensalt())
```

bcrypt.gensalt():

- Generates a **random 128-bit salt** internally
- Encodes it in bcrypt's own Base64 variant
- Embeds it inside the output string

The result looks like:

```
$2b$12$abcdefghijklmnopqrstuv12345678901234567890123456
```

This string contains:

- `$2b$` → bcrypt version
- `12` → cost factor
- salt (22 chars)

- hash (31 chars)

Where is the salt in a bcrypt hash?

`$2b$12$Gxl4Zij4gqX7GpeX0sE3e.TK9dL5./xJrpQHUNRw9eEmq3T5jl9.i`

Breakdown:

Part	Meaning
<code>\$2b\$</code>	bcrypt version
<code>12</code>	cost (work factor)
<code>\$Gxl4Zij4gqX7GpeX0sE3e.</code>	Salt (22 chars)
<code>TK9dL5./xJrpQHUNRw9eEmq3T5jl9.i</code>	Hash

So: `salt = bcrypt_hash[:29].decode()`

Extracts up to the end of the salt area.

Why bcrypt does this automatically

bcrypt is designed so that:

- The salt is not stored separately
- The hash contains everything needed
- Verification is always possible without external salt

`bcrypt.checkpw(password.encode(), bcrypt_hash)`

bcrypt decodes:

- The cost
- The salt
- And recomputes the hash

All from the encoded string.

script Block

Using:

$n = 2 ** 14$ (*CPU cost*)

$r = 8$ (*block size*)

$p = 1$ (*parallel*)

scrypt is memory hard.

Same measurements applied.

```
scrypt_hash = hashlib.scrypt(
    password.encode(),
    salt=salt,
    n=scrypt_n,
    r=scrypt_r,
    p=scrypt_p
)
```

Here:

- *password.encode()* → *the password*
- *salt = salt* → *16 – byte random salt*
- *n,r,p* → *memory and cost parameters*

```
scrypt_verify = (
    hashlib.scrypt(password.encode(), salt=salt, n=..., r=...,
p=...)

    == scrypt_hash
)
```

Verify hashes for the *same salt* again.

Argon2 Block

Argon2id parameters:

time_cost = 2 iterations

memory_cost = 102400 KiB (= 100 MB)

parallelism = 8

These are **strong parameters**.

Argon2 stores:

- salt
- time/memory parameters
- hash

in one encoded string.

Verification uses the internal salt.

1. What Argon2 Is

Argon2 is the **winner of the Password Hashing Competition (PHC)** and is now a NIST-recommended modern password hashing algorithm.

There are 3 variants:

- **Argon2i** – resistant to side-channel attacks
- **Argon2d** – resistant to GPU cracking
- **Argon2id** – hybrid (recommended and what the python library uses)

Our library:

```
from argon2 import PasswordHasher
```

Uses **Argon2id**.

2. What the Salt Does in Argon2

Argon2 **always embeds its salt inside the final hash string**.

Unlike SHA256 or scrypt, Argon2 uses:

- A **random salt generated automatically**

- Typically **16 bytes**
- Mixed in the internal memory-hard blocks

Salt ensures:

- Same password always → different hash
- Prevents rainbow tables
- Forces attackers to re-compute Argon2 for each salt

Even if you hash the same password 1 million times, the salt randomness prevents patterns.

The code:

```
hasher = PasswordHasher(time_cost=2, memory_cost=102400,
parallelism=8)

argon_hash = hasher.hash(password)
```

What happens:

Step 1 Generate random salt

Argon2 automatically creates a new random salt (16 bytes typical).

Step 2 — Run Argon2id using:

- ***time_cost = 2***
→ How many iterations / passes over memory
- ***memory_cost = 102400***
→ Memory usage in KiB → **102,400 KiB = 100 MB**
- ***parallelism = 8***
→ 8 lanes of parallel filling

This means:

- Argon2 call forces a minimum 100 MB memory usage per hash
- GPU cracking becomes extremely expensive.

3. Internal Steps Argon2 Executes (Technical)

A full Argon2 hash consists of:

(1) Initial hash H_0

Argon2 first builds an initial hash from:

- password
- salt
- memory_cost
- time_cost
- parallelism
- version
- Argon2 type (i, d, id)

This H_0 defines *everything* for the memory-filling phase

(2) Build Memory Matrix

Argon2 allocates a large memory region:

memory_cost blocks

each block = 1024 bytes

Our implementation:

100 MB → around 100,000 blocks

Then Argon2 splits memory into **parallel lanes** (8 in our case).

(3) Fill memory using BLAKE2b-based compression

Argon2 fills blocks using a complex function:

$Block[i] = G(Block[i - 1], RandomlySelectedPreviousBlock)$

The random selection depends on:

- password
- salt
- pass number
- block index

This creates **data-dependent memory access**.

This is what kills GPU performance.

(4) Second pass (time_cost)

We specify:

time_cost = 2

Meaning:

Argon2 makes **two full sweeps** over the 100 MB memory region.

(5) Final extraction

After filling memory, Argon2 compresses all lane results into a final BLAKE2b digest → the final hash.

4. What our Argon2 Hash String Contains

Our returned hash string looks like:

\$argon2id\$v = 19\$m = 102400,t = 2,p = 8\$ < base64salt > \$ < base64hash >

This encodes:

- algorithm (*argon2id*)
- version (*v* = 19)
- memory cost (*m* = 102400)
- time cost (*t* = 2)
- parallelism (*p* = 8)
- salt (*base64*)
- final hash output (*base64*)

Everything needed for verification is inside one string.

5. How Verification Works

The code:

hasher.verify(argon_hash, password)

What happens:

The library parses the hash string

Extracts:

- memory_cost
- time_cost
- parallelism
- salt

Re-runs **Argon2id** with same parameters and same salt

Compares computed hash vs stored hash

If equal → verification succeeds.

6. Why Argon2 is perfect as the Final Layer of this Pipeline

The pipeline is:

Password

→ ***Plot – based chaotic transformation***

→ ***Per – character functions***

→ ***Avalanche mixing***

→ ***Normalization***

→ ***SHA256 binary mixing***

→ ***Argon2***

Argon2 is ideal because:

- It is **memory-hard**

The 100 MB setting forces attackers to use massive RAM.

- It **neutralizes any remaining patterns**

Even if the earlier plot-layer produced structured bits,
Argon2's memory-filling phase destroys structure.

- It **makes brute-force extremely expensive**

Every guess requires:

- allocating 100 MB
- doing 2 full memory passes
- running 8-lane compression
- computing Blake2b many times

Attackers must spend **RAM × time cost per guess**.

- It **works on binary inputs**

The *final_binary* from the plot-layer is NOT a password its random floats serialized into bytes.
Argon2 fully accepts arbitrary bytes.

final_binary = 32 bytes

The pipeline produces a raw binary and fed into the argon2.

This *final_binary* is:

- `SHA256(binary_data).digest()`
- always 32 raw bytes
- not a string
- not user-readable
- contains high entropy
- contains nonlinear float transformations

So it **is raw binary**, the exact thing Argon2 was designed to accept.

argon2(SHA256(transformed_plot_based_binary_data))

This input:

- Has no human meaning
- Far higher entropy
- Predictable length (32 bytes → ideal)
- No dictionary match
- No password-structure leakage
- No patterns

- No Unicode/charset issues
- No low-entropy weaknesses

So Argon2 only needs to do **one job**:

- Make the attack economically infeasible
- By using memory hardness + parallelism resistance.

This makes the pipeline:

- deterministic
- irreversible
- extremely slow for attackers
- resistant to GPU/ASIC/FPGA cracking
- resistant to time-memory tradeoff attacks

The entire design can be summarized as:

***Password → Plot Layer → nonlinear chaotic transformation → SHA256
→ 32 bytes → Argon2 → final hash***